32-bit Modular Arithmetic and SIMD Vectorization

Duc Vinh Nguyen, Henry Zheng

March 2026

SORBONNE UNIVERSITÉ

Contents

1 Introduction **2**

2 Fundamental Cost of Modular Reduction **2**

3 Shoup's Modular Multiplication **2**

3.1 Input Constraint: . . . 2
3.2 Theorem . . . 3
3.3 Input Hypothesis: . . . 3
3.4 Precomputational Logic (Barrett Reduction) . . . 3
3.5 Fundamental Principle . . . 4
3.6 Demonstration: . . . 4
3.7 Advantages . . . 5
3.8 SIMD vectorization Implementation . . . 5
3.8.1 AVX2 . . . 6
3.8.2 AVX-512 . . . 11
3.8.3 NEON . . . 14
3.9 Performance . . . 16
3.9.1 Overall Hypothesis . . . 16
3.9.2 Equipment details for AMD architecture . . . 16
3.9.3 Equipment details for ARM architecture . . . 16
3.9.4 Choice of unrolling . . . 16
3.9.5 Choice between mullo_epi32 and mul_epu32 . . . 18
3.9.6 Versions of mullo: . . . 19
3.9.7 Overall Performance . . . 20
3.9.8 NEON's Performance . . . 22

4 Overall Conclusion: **22**

5 Other Improvements: **22**

1 Introduction

This project explores and optimizes modular reductions modulo primes $p < 2^{32}$ using SIMD vectorization (AVX2, AVX-512, NEON) along with various techniques and algorithms.

Objective: The aim of this study is to develop an optimized implementation of modular reduction for 32-bit primes, minimizing execution time through SIMD vectorization.

2 Fundamental Cost of Modular Reduction

The naive approach to computing $a \pmod{n}$ is to evaluate:

$$a - \left\lfloor \frac{a}{n} \right\rfloor \cdot n \tag{1}$$

However, on modern processor architectures (Intel, AMD,...), integer division has a significantly higher computational cost compared to basic arithmetic operations such as additions and multiplications.

Since equation (1) requires an integer division, a multiplication and a subtraction operation, it is considered highly inefficient from both a software and hardware perspective.

3 Shoup's Modular Multiplication

Shoup's algorithm is based on precomputation to avoid costly operations.

Algorithm 1: Shoup's modular multiplication algorithm

Input : $A, B \in [0, p[$, we suppose that $p < \beta/2$
precompute $B' = \lfloor B\beta/p \rfloor$

Output: $R = AB \bmod p$

```
Q ← ⌊AB'/β⌋ ;
R ← (AB − Qp) mod β ;
if R ≥ p then
    R ← R − p;
```

3.1 Input Constraint:

To ensure the correctness of the modular reduction and prevent register overflow during SIMD operations, the following conditions must be satisfied:

- **Word Size (β):** Set to 2^{32} for 32-bit modular arithmetic
- **Input 1 (A):** An integer defined over 32 bits
- **Input 2 (B):** Integer within the range $[0, p[$
- **Modulus (p):** A prime number defined over 31 bits ($p < 2^{31}$)

3.2 Theorem

Let p be a prime number such that $p < \beta/2$ where $\beta = 2^{32}$. Given an integer A defined over 32 bits and $B \in [0, p)$, let B' be the precomputed value $B' = \lfloor B\beta/p \rfloor$. If the estimated quotient is defined as $Q = \lfloor AB'/\beta \rfloor$, then the intermediate remainder $R = AB - Qp$ satisfies the following condition:

$$0 \leq AB - Qp < 2p$$

3.3 Input Hypothesis:

Can p be any integer over 32 bits ?
No, one particularly interesting case where Shoup's algorithm will fail is for $p \geq 2^{31}$. Since we know that:

$$0 \leq AB - Qp < 2p$$

If $p \geq 2^{31}$, then $2p \geq 2^{32}$. Therefore, in some cases $AB - Qp$ may exceed 2^{32}. This causes tremendous errors in the calculation because all bits beyond 32 bits are destroyed when we reduce modulo β

Does p have to necessarily be a prime number ?
No, the algorithm mainly uses precomputations (Barrett Reduction), optimized for calculation of $a \mod n$ without constraints on n. But for the sake of this project, n is a prime number (p).

Can b be any integer over 32 bits ?
No, suppose $b > p$, we can rewrite b as:

$$b = q' \cdot p + r'$$

Thus the calculation for the precomputed value B' is:

$$B' = \left\lfloor \frac{(q' \cdot p + r') \cdot 2^{32}}{p} \right\rfloor$$

$$B' = q' \cdot 2^{32} + \left\lfloor \frac{r' \cdot 2^{32}}{p} \right\rfloor$$

With a focus on 32 bits in this project, B' in the case of $b > p$ cannot be stored within 32 bits and will overflow.

Can a be any integer over 32 bits ?
Yes, a can be any integer defined over 32 bits.

3.4 Precomputational Logic (Barrett Reduction)

The precomputed value B' is defined over 32 bits as:

$$B' = \lfloor (B \cdot 2^{32})/p \rfloor$$

When computing the $A \cdot B \pmod p$:

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot B}{p} \right\rfloor \cdot p$$

We can rewrite this:

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot \left\lfloor \left(\frac{B \cdot 2^{32}}{p}\right) \right\rfloor}{2^{32}} \right\rfloor \cdot p$$
$$\approx A \cdot B - \left\lfloor \frac{A \cdot B'}{2^{32}} \right\rfloor \cdot p$$

3.5 Fundamental Principle

Given x, the quotient of two 32-bit integers:
The proof of this algorithm is based on the non-decimal value of ($\lfloor x \rfloor$):

$$x - 1 < \lfloor x \rfloor \leq x$$

We can rewrite this as:

$$0 \leq x - \lfloor x \rfloor < 1$$

3.6 Demonstration:

The algorithm uses the precomputation $B' = \lfloor B\beta/p \rfloor$ such that:

$$0 \leq \frac{B\beta}{p} - \left\lfloor \frac{B\beta}{p} \right\rfloor < 1$$
$$0 \leq \frac{B\beta}{p} - B' < 1 \tag{2}$$

Using the same logic, the algorithm estimates the quotient $Q = \lfloor AB'/\beta \rfloor$ such that:

$$0 \leq \frac{AB'}{\beta} - \left\lfloor \frac{AB'}{\beta} \right\rfloor < 1$$
$$0 \leq \frac{AB'}{\beta} - Q < 1 \tag{3}$$

We now prove that the remainder $R = (A \cdot B - Q \cdot p)$ lies within the interval $[0, 2p[$:

We first multiply the equation (1) by Ap/β:

$$0 \leq \left(\frac{B\beta}{p} - B'\right) \cdot \frac{Ap}{\beta} < 1 \cdot \frac{Ap}{\beta}$$
$$0 \leq AB - \frac{AB'p}{\beta} < \frac{Ap}{\beta} \tag{4}$$

Similarly, we multiply the equation (2) by p:

$$0 \leq \left(\frac{AB'}{\beta} - Q\right) \cdot p < 1 \cdot p$$

$$0 \leq \frac{AB'p}{\beta} - Qp < p \tag{5}$$

By adding equations (3) and (4), we deduce:

$$0 \leq (AB - \cancel{\frac{AB'p}{\beta}}) + (\cancel{\frac{AB'p}{\beta}} - Qp) < \frac{Ap}{\beta} + p$$

Simplifying this result, we are left with:

$$0 \leq AB - Qp < p + \frac{Ap}{\beta}$$
$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta})$$

By definition, $A \in [0, p[$ and $p < \beta$, thus $A < \beta$. Therefore we can write:

$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta}) < p \cdot (1 + \frac{\beta}{\beta})$$
$$0 \leq AB - Qp < p \cdot (1 + 1)$$
$$0 \leq AB - Qp < 2p$$
$$0 \leq R < 2p$$

Conclusion: We have now proven that $R \in [0, 2p[$. The final if statement of the algorithm helps correct the remainder: if $R \geq p$, one more step $R \leftarrow R - p$ is needed; if not, R is the correct result.

3.7 Advantages

Using the precomputed value B' to approximately calculate $\left\lfloor \frac{A \cdot B}{p} \right\rfloor$ improves upon the naive method in two ways:

1. **Storing the precomputed value:** B' is computed with one division and can be stored for use in other modular reductions, provided the same B and the same p are used.
2. **Efficient operations:** Shoup's algorithm replaces slow division operations with fast multiplications and bit shifts. The expression $\frac{A \cdot B'}{2^{32}}$ requires only one multiplication operation and a right bit shift by 32 bits. On a CPU, both multiplication and bit shift operations are significantly more efficient than division.

3.8 SIMD vectorization Implementation

We decided to create three versions for implementing a high-performance Shoup algorithm: AVX2, AVX-512 and NEON.

3.8.1 AVX2

For the AVX2 version, we decided to use two main AVX2 functions: `_mm256_mul_epu32` and `_mm256_mullo_epi32`.

Intel `_mm256_mul_epu32`
This function treats the input as unsigned 32-bit integers. However, it only multiplies the even-indexed slots (0, 2, 4, 6) of the vector to make room for the full 64-bit results.

This implementation uses purely `_mm256_mul_epu32` at all stages of calulations thus we must data shuffle the registers to their correct position. The following are the steps per loop:

- Load 8 32 bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64 bit registers
- Compute Shoup's Modular Reduction (`_shoup_avx2(...)`) over these registers
- Recombine the even and odd indexed slots and store
- Compute the leftover elements

Listing 1: Auxiliary Function of Shoup's Modular Reduction

```
static inline __m256i _shoup_avx2(__m256i va,
                                  __m256i vb,
                                  __m256i vb_precomp,
                                  __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // vab = [ (a_0 * b_0 = ab_0ab_1), (a_2 * b_2 = ab_2ab_3),
    // (a_4 * b_4 = ab_4ab_5), (a_6 * b_6 = ab_6ab_7) ]
    __m256i vab = _mm256_mul_epu32(va, vb);

    // vqp = [ (q_0 * p_0 = qp_0qp_1), (q_2 * p_2 = qp_2qp_3),
    // (q_4 * p_4 = qp_4qp_5), (q_6 * p_6 = qp_6qp_7) ]
    __m256i vqp = _mm256_mul_epu32(vq, vp);

    // vc = [ (ab_0 - qp_0 = c_0c_1), (ab_2 - qp_2 = c_2c_3),
    // (ab_4 - qp_4 = c_4c_5), (ab_6 - qp_6 = c_6c_7) ]
    __m256i vc = _mm256_sub_epi64(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 2: AVX2 Implementation of Shoup's Modular Reduction

```
Vector shoup_scale_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
```

```
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        // Load [ a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7 ]
        //    = [ (a_1, a_0), (a_3, a_2), (a_5, a_4), (a_7, a_6) ]
        // but only even index will be directly handled
        __m256i even_va = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i)
        );

        // Need to move odd index to even index to be handled:
        // [ (0, a_1), (0, a_3), (0, a_5), (0, a_7) ]
        __m256i odd_va = _mm256_srli_epi64(even_va, 32);

        // Process Even Lanes
        __m256i even_vc = _shoup_avx2(even_va, vb, vb_precomp, vp);

        // Process Odd Lanes:
        __m256i odd_vc = _shoup_avx2(odd_va, vb, vb_precomp, vp);

        // Shift odd by 32 bits to the left results back and merge:
        // [ (c_1, 0), (c_3, 0), (c_5, 0), (c_7, 0) ]
        __m256i odd_shifted = _mm256_slli_epi64(odd_vc, 32);

        // Merge even and odd: [ c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7 ]
        __m256i merge = _mm256_or_si256(even_vc, odd_shifted);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Intel `_mm256_mullo_epi32`
This function treats the input as signed 32-bit integers. It multiplies all 8 pairs of numbers but only stores the bottom half of the result.

This implementation uses `_mm256_mul_epu32` to compute the quotient:

$$q = \lfloor (a \cdot b_{precomp})/2^{32} \rfloor$$

but `_mm256_mullo_epi32` to compute all other steps per loop. The following are the steps per loop:

- Load 8 32 bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64 bit registers
- Compute Shoup's Modular Reduction (`_shoup_mullo_avx2(...)`) over these registers
- Recombine the even and odd indexed slots and store
- Compute the leftover elements

Listing 3: Auxiliary Function of Shoup's Modular Reduction (Version with multiply-low)

```
static inline __m256i _shoup_mullo_avx2(__m256i va,
                                        __m256i vb,
                                        __m256i vb_precomp,
                                        __m256i vp)
{
    // vq = [ (a_0 * b_precomp_0 = q_0q_1), (a_2 * b_precomp_2 = q_2q_3),
    // (a_4 * b_precomp_4 = q_4q_5), (a_6 * b_precomp_6 = q_6q_7) ]
    __m256i vq = _mm256_mul_epu32(va, vb_precomp);

    // vq = [ (0, q_0), (0, q_2), (0, q_4), (0, q_6) ]
    vq = _mm256_srli_epi64(vq, 32);

    // Preserves only the least significant bit:
    // vab = [ (a_0 * b_0 mod 2^32 = ab_0), (a_2 * b_2 mod 2^32 = ab_2),
    // (a_4 * b_4 mod 2^32 = ab_4), (a_6 * b_6 mod 2^32 = ab_6) ]
    __m256i vab = _mm256_mullo_epi32(va, vb);

    // vqp = [ [q_0 * p_0 mod 2^32 = qp_0], [q_2 * p_2 mod 2^32 = qp_2],
    // [q_4 * p_4 mod 2^32 = qp_4], [q_6 * p_6 mod 2^32 = qp_6] ]
    __m256i vqp = _mm256_mullo_epi32(vq, vp);

    // vc = [ (0, ab_0 - qp_0), (0, ab_2 - qp_2), (0, ab_4 - qp_4),
    // (0, ab_6 - qp_6) ]
    __m256i vc = _mm256_sub_epi32(vab, vqp);

    // if (c >= p) c -= p
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 4: AVX2 Implementation of Shoup's Modular Reduction (Version with multiply-low)

```
Vector shoup_scale_mullo_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
```

```
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    ulong i = 0;
    for (; i + 31 < size; i += 32)
    {
        __m256i even_va_0 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i));
        __m256i even_va_1 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 8));
        __m256i even_va_2 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 16));
        __m256i even_va_3 = _mm256_loadu_si256(
            (__m256i const *)(v.elements + i + 24));

        __m256i odd_va_0 = _mm256_srli_epi64(even_va_0, 32);
        __m256i odd_va_1 = _mm256_srli_epi64(even_va_1, 32);
        __m256i odd_va_2 = _mm256_srli_epi64(even_va_2, 32);
        __m256i odd_va_3 = _mm256_srli_epi64(even_va_3, 32);

        __m256i even_vc_0 = _shoup_mullo_avx2(even_va_0, vb, vb_precomp, vp);
        __m256i even_vc_1 = _shoup_mullo_avx2(even_va_1, vb, vb_precomp, vp);
        __m256i even_vc_2 = _shoup_mullo_avx2(even_va_2, vb, vb_precomp, vp);
        __m256i even_vc_3 = _shoup_mullo_avx2(even_va_3, vb, vb_precomp, vp);

        __m256i odd_vc_0 = _shoup_mullo_avx2(odd_va_0, vb, vb_precomp, vp);
        __m256i odd_vc_1 = _shoup_mullo_avx2(odd_va_1, vb, vb_precomp, vp);
        __m256i odd_vc_2 = _shoup_mullo_avx2(odd_va_2, vb, vb_precomp, vp);
        __m256i odd_vc_3 = _shoup_mullo_avx2(odd_va_3, vb, vb_precomp, vp);

        __m256i odd_shifted_0 = _mm256_slli_epi64(odd_vc_0, 32);
        __m256i odd_shifted_1 = _mm256_slli_epi64(odd_vc_1, 32);
        __m256i odd_shifted_2 = _mm256_slli_epi64(odd_vc_2, 32);
        __m256i odd_shifted_3 = _mm256_slli_epi64(odd_vc_3, 32);

        __m256i merge_0 = _mm256_or_si256(even_vc_0, odd_shifted_0);
        __m256i merge_1 = _mm256_or_si256(even_vc_1, odd_shifted_1);
        __m256i merge_2 = _mm256_or_si256(even_vc_2, odd_shifted_2);
        __m256i merge_3 = _mm256_or_si256(even_vc_3, odd_shifted_3);

        _mm256_storeu_si256((__m256i *)(res.elements + i), merge_0);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 8), merge_1);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 16), merge_2);
        _mm256_storeu_si256((__m256i *)(res.elements + i + 24), merge_3);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Intel `_mm256_mullo_epi32` version 2:
Derived from the previous version, this version aims to regroup the vector registers immediately after the multiplication step:

$$q = \lfloor (a \cdot b_{precomp})/2^{32} \rfloor$$

By restoring the standard 8-lane 32-bit layout early, the function eliminates the need to

perform parallel logic paths. This, in theory, should reduce the workload.

In order to implement this version, we create an Auxiliary function **_mulhi_emul_avx2** which emulates a native 'multiply-high' instruction for unsigned 32-bit integers. It computes the full 64-bit products of two packed vectors, extracts the most significant 32 bits from each result, and repacks them into a single 256-bit destination register.

The following are the steps per loop:

- Load 8 32 bit integers into the register
- Split the loaded register into even-indexed slots (0, 2, 4, 6) and odd-indexed slots (1, 3, 5, 7) into 64 bit registers to compute the quotient
- Recombine the even and odd indexed slots
- Continue with Shoup's Modular Reduction (`_shoup_mullo_v2_avx2(...)`)
- Compute the leftover elements

Listing 5: Auxiliary Function `_mulhi_emul_avx2`

```
static inline __m256i _mulhi_emul_avx2(__m256i va, __m256i vb)
{
    // res_even = [ (a_0 * b_0), (a_2 * b_2), (a_4 * b_4), (a_6 * b_6) ]
    __m256i res_even = _mm256_mul_epu32(va, vb);

    // res_odd = [ (a_1 * b_0), (a_3 * b_2), (a_5 * b_4), (a_7 * b_6) ]
    __m256i res_odd = _mm256_mul_epu32(_mm256_srli_epi64(va, 32), vb);

    // hi_even = [ (0, hi(a_0 * b_0)), (0, hi(a_2 * b_2)), (0, hi(a_4 * b_4)),
    // (0, hi(a_6*b_6)) ]
    __m256i hi_even = _mm256_srli_epi64(res_even, 32);

    // Removes the least significant bits: hi_odd = [ (hi(a_1 * b_0), 0),
    // (hi(a_3 * b_2), 0), (hi(a_5 * b_4), 0), (hi(a_7 * b_6), 0) ]
    __m256i hi_odd = _mm256_and_si256(res_odd, _mm256_set1_epi64x(
        0xFFFFFFFF00000000ULL));

    // [ (hi(a_1 * b_0), hi(a_0 * b_0)), (hi(a_3 * b_2), hi(a_2 * b_2)),
    // (hi(a_5 * b_4), hi(a_4 * b_4)), (hi(a_7 * b_6), hi(a_6 * b_6)) ]
    return _mm256_or_si256(hi_even, hi_odd);
}
```

Listing 6: Auxiliary Function of Shoup's Modular Reduction (Version 2 of multiply-low)

```
static inline __m256i _shoup_mullo_v2_avx2(__m256i va,
                                           __m256i vb,
                                           __m256i vb_precomp,
                                           __m256i vp)
{
    __m256i vq = _mulhi_emul_avx2(va, vb_precomp);
    __m256i vab = _mm256_mullo_epi32(va, vb);
    __m256i vqp = _mm256_mullo_epi32(vq, vp);
    __m256i vc = _mm256_sub_epi32(vab, vqp);
    return _mm256_min_epu32(vc, _mm256_sub_epi32(vc, vp));
}
```

Listing 7: AVX2 Implementation of Shoup's Modular Reduction (Version 2 of multiply-low)

```
Vector shoup_scale_mullo_v2_avx2(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi32(param.b);
    __m256i vb_precomp = _mm256_set1_epi32(param.b_precomp);
    __m256i vp = _mm256_set1_epi32(param.p);
    ulong i = 0;
    for (; i + 7 < size; i += 8)
    {
        __m256i va = _mm256_loadu_si256((__m256i const *)(v.elements + i));

        __m256i vc = _shoup_mullo_v2_avx2(va, vb, vb_precomp, vp);
        _mm256_storeu_si256((__m256i *)(res.elements + i), vc);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

3.8.2 AVX-512

Listing 8: Auxiliary Function of Shoup's Modular Reduction

```
static inline __m512i _shoup_avx512(__m512i va,
                                    __m512i vb,
                                    __m512i vb_precomp,
                                    __m512i vp)
{
    __m512i vq = _mm512_mul_epu32(va, vb_precomp);
    vq = _mm512_srli_epi64(vq, 32);
    __m512i vab = _mm512_mul_epu32(va, vb);
    __m512i vqp = _mm512_mul_epu32(vq, vp);
    __m512i vc = _mm512_sub_epi64(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 9: AVX-512 Implementation of Shoup's Modular Reduction

```
Vector shoup_scale_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi64(param.b);
    __m512i vb_precomp = _mm512_set1_epi64(param.b_precomp);
    __m512i vp = _mm512_set1_epi64(param.p);
    ulong i = 0;
    for (; i + 15 < size; i += 16)
    {
        __m512i even_va = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i));

        __m512i odd_va = _mm512_srli_epi64(even_va, 32);

```

```
        __m512i even_vc = _shoup_avx512(even_va, vb, vb_precomp, vp);

        __m512i odd_vc = _shoup_avx512(odd_va, vb, vb_precomp, vp);

        __m512i odd_shifted = _mm512_slli_epi64(odd_vc, 32);

        __m512i merge = _mm512_or_si512(even_vc, odd_shifted);

        _mm512_storeu_si512((__m512 *)(res.elements + i), merge);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 10: Auxiliary Function of Shoup's Modular Reduction (Version with multiply-low)

```
static inline __m512i _shoup_mullo_avx512(__m512i va,
                                          __m512i vb,
                                          __m512i vb_precomp,
                                          __m512i vp)
{
    __m512i vq = _mm512_mul_epu32(va, vb_precomp);
    vq = _mm512_srli_epi64(vq, 32);
    __m512i vab = _mm512_mullo_epi32(va, vb);
    __m512i vqp = _mm512_mullo_epi32(vq, vp);
    __m512i vc = _mm512_sub_epi32(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 11: AVX-512 Implementation of Shoup's Modular Reduction (Version with multiply-low)

```
Vector shoup_scale_mullo_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi64(param.b);
    __m512i vb_precomp = _mm512_set1_epi64(param.b_precomp);
    __m512i vp = _mm512_set1_epi64(param.p);
    ulong i = 0;
    for (; i + 63 < size; i += 64)
    {
        __m512i even_va_0 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i));
        __m512i even_va_1 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 16));
        __m512i even_va_2 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 32));
        __m512i even_va_3 = _mm512_loadu_si512(
            (__m512i const *)(v.elements + i + 48));

        __m512i odd_va_0 = _mm512_srli_epi64(even_va_0, 32);
        __m512i odd_va_1 = _mm512_srli_epi64(even_va_1, 32);
        __m512i odd_va_2 = _mm512_srli_epi64(even_va_2, 32);
        __m512i odd_va_3 = _mm512_srli_epi64(even_va_3, 32);
```

```

        __m512i even_vc_0 = _shoup_mullo_avx512(even_va_0,
                                                vb,
                                                vb_precomp,
                                                vp);
        __m512i even_vc_1 = _shoup_mullo_avx512(even_va_1,
                                                vb,
                                                vb_precomp,
                                                vp);
        __m512i even_vc_2 = _shoup_mullo_avx512(even_va_2,
                                                vb,
                                                vb_precomp,
                                                vp);
        __m512i even_vc_3 = _shoup_mullo_avx512(even_va_3,
                                                vb,
                                                vb_precomp,
                                                vp);

        __m512i odd_vc_0 = _shoup_mullo_avx512(odd_va_0, vb, vb_precomp, vp);
        __m512i odd_vc_1 = _shoup_mullo_avx512(odd_va_1, vb, vb_precomp, vp);
        __m512i odd_vc_2 = _shoup_mullo_avx512(odd_va_2, vb, vb_precomp, vp);
        __m512i odd_vc_3 = _shoup_mullo_avx512(odd_va_3, vb, vb_precomp, vp);

        __m512i odd_shifted_0 = _mm512_slli_epi64(odd_vc_0, 32);
        __m512i odd_shifted_1 = _mm512_slli_epi64(odd_vc_1, 32);
        __m512i odd_shifted_2 = _mm512_slli_epi64(odd_vc_2, 32);
        __m512i odd_shifted_3 = _mm512_slli_epi64(odd_vc_3, 32);

        __m512i merge_0 = _mm512_or_si512(even_vc_0, odd_shifted_0);
        __m512i merge_1 = _mm512_or_si512(even_vc_1, odd_shifted_1);
        __m512i merge_2 = _mm512_or_si512(even_vc_2, odd_shifted_2);
        __m512i merge_3 = _mm512_or_si512(even_vc_3, odd_shifted_3);

        _mm512_storeu_si512((__m512 *)(res.elements + i), merge_0);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 16), merge_1);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 32), merge_2);
        _mm512_storeu_si512((__m512 *)(res.elements + i + 48), merge_3);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 12: Listing 5: Auxiliary Function _mulhi_emul_avx512

```
static inline __m512i _mulhi_emul_avx512(__m512i va, __m512i vb)
{
    __m512i res_even = _mm512_mul_epu32(va, vb);
    __m512i res_odd = _mm512_mul_epu32(_mm512_srli_epi64(va, 32), vb);
    __m512i hi_even = _mm512_srli_epi64(res_even, 32);
    __m512i hi_odd = _mm512_and_si512(res_odd,
                                      _mm512_set1_epi64(
                                        0xFFFFFFFF00000000ULL));
    return _mm512_or_si512(hi_even, hi_odd);
}
```

Listing 13: Auxiliary Function of Shoup's Modular Reduction (Version 2 of multiply-low)

```
static inline __m512i _shoup_mullo_v2_avx512(__m512i va,
                                             __m512i vb,
                                             __m512i vb_precomp,
                                             __m512i vp)
{
    __m512i vq = _mulhi_emul_avx512(va, vb_precomp);
    __m512i vab = _mm512_mullo_epi32(va, vb);
    __m512i vqp = _mm512_mullo_epi32(vq, vp);
    __m512i vc = _mm512_sub_epi32(vab, vqp);
    return _mm512_min_epu32(vc, _mm512_sub_epi32(vc, vp));
}
```

Listing 14: AVX-512 Implementation of Shoup's Modular Reduction (Version 2 of multiply-low)

```
Vector shoup_scale_mullo_v2_avx512(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    __m512i vb = _mm512_set1_epi32(param.b);
    __m512i vb_precomp = _mm512_set1_epi32(param.b_precomp);
    __m512i vp = _mm512_set1_epi32(param.p);
    ulong i = 0;
    for (; i + 15 < size; i += 16)
    {
        __m512i va = _mm512_loadu_si512((__m512i const *)(v.elements + i));

        __m512i vc = _shoup_mullo_v2_avx512(va, vb, vb_precomp, vp);
        _mm512_storeu_si512((__m512i *)(res.elements + i), vc);
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

3.8.3 NEON

Listing 15: Auxiliary Function of Shoup's Modular Reduction (Version Neon)

```
static inline uint32x2_t _shoup_neon(uint32x2_t va,
                                     uint32x2_t vb,
                                     uint32x2_t vb_precomp,
                                     uint32x2_t vp)
{
    // vab_precomp = [ a_0 * b_precomp_0 = ab_precomp_0,
    // a_1 * b_precomp_1 = ab_precomp_1 ]
    uint64x2_t vab_precomp = vmull_u32(va, vb_precomp);

    // vq = [ q_0, q_1 ]
    uint32x2_t vq = vshrn_n_u64(vab_precomp, 32);

    // vab = [ a_0 * b_0 = ab_0, a_1 * b_1 = ab_1 ]
    uint64x2_t vab = vmull_u32(va, vb);

```

```
    // vqp = [ q_0 * p_0 = qp_0, q_1 * p_1 = qp_1 ]
    uint64x2_t vqp = vmull_u32(vq, vp);

    // vc = [ ab_0 - qp_0 = c_0, ab_1 - qp_1 = c_1 ]
    uint32x2_t vc = vmovn_u64(vsubq_u64(vab, vqp));

    // if (c >= p) c -= p
    return vmin_u32(vc, vsub_u32(vc, vp));
}
```

Listing 16: NEON Implementation of Shoup's Modular Reduction

```
Vector shoup_scale_neon(Parameters param, Vector v)
{
    ulong size = v.size;
    Vector res = init_vector(size);
    ulong n = size - (size % 4);
    uint32x2_t vb = vdup_n_u32(param.b);
    uint32x2_t vp = vdup_n_u32(param.p);
    uint32x2_t vb_precomp = vdup_n_u32(param.b_precomp);
    ulong i = 0;
    for (; i + 1 < n; i += 2)
    {
        // Load [ a_0, a_1 ]
        uint32x2_t va = vld1_u32(v.elements + i);

        // Stocks the value
        vst1_u32(res.elements + i, _shoup_neon(va, vb, vb_precomp, vp));
    }
    for (; i < size; i++)
        *(res.elements + i) = shoup(*(v.elements + i),
                                    param.b,
                                    param.b_precomp,
                                    param.p);
    return res;
}
```

Listing 17: Auxiliary Function of Shoup's Modular Reduction (Version with multiply-low)

```
static inline uint32x2_t _shoup_mullo_neon(uint32x2_t va,
                                           uint32x2_t vb,
                                           uint32x2_t vb_precomp,
                                           uint32x2_t vp)
{
    // vab_precomp = [ a_0 * b_precomp_0 = ab_precomp_0,
    // a_1 * b_precomp_1 = ab_precomp_1 ]
    uint64x2_t vab_precomp = vmull_u32(va, vb_precomp);

    // vq = [ q_0, q_1 ]
    uint32x2_t vq = vshrn_n_u64(vab_precomp, 32);

    // vab = [ a_0 * b_0 mod 2^32 = ab_0, a_1 * b_1 mod 2^32 = ab_1 ]
    uint32x2_t vab = vmul_u32(va, vb);

    // vqp = [ q_0 * p_0 mod 2^32 = qp_0, q_1 * p_1 mod 2^32 = qp_1 ]
    uint32x2_t vqp = vmul_u32(vq, vp);

    // vc = [ ab_0 - qp_0 = c_0, ab_1 - qp_1 = c_1]
    uint32x2_t vc = vsub_u32(vab, vqp);
    return vmin_u32(vc, vsub_u32(vc, vp));
```

```
}
```

3.9 Performance

The performance benchmarks were run on AMD and ARM architectures, by executing `./pcca7 -b_bits 31`.

3.9.1 Overall Hypothesis

Using SIMD operations should provide us with a max acceleration of 8x-16x since the registers can hold and operate on 8 32 bit unsigned integers (for AVX2) and 16 32 bit unsigned integers (for AVX512) simultanously.

One caveate that must be considered is that all implementation uses `_mm256_mul_epu32` to calculate the quotient defined as $Q = \lfloor AB'/\beta \rfloor$. This operation requires a data shuffle to correctly align the vectors into 64 bit lanes then recombining them at later stages. This reduces the acceleration factor of SIMD implementations at worst by half (4x-8x depending on AVX2 or AVX512)

3.9.2 Equipment details for AMD architecture

- Processor: AMD Ryzen™ 5 8600G Desktop Processor.
- Instruction Sets Supported: AVX, AVX2 and AVX-512
- Compiler: GCC 13.3.0 with optimization flag `-O3`

3.9.3 Equipment details for ARM architecture

- Processor: Apple M1
- Instruction Sets Supported: NEON
- Compiler: GCC 13.3.0 with optimization flag `-O3`

Note: The benchmarks were run in a virtual machine.

3.9.4 Choice of unrolling

One way to optimize for faster functions through vectorization is Loop Unrolling. This technique consists of expanding the body of a loop to process multiple iterations at once.

The results below were obtained by measuring the execution time between a standard version and unrolled version of Listing 2: AVX2 Implementation of Shoup's Modular Reduction

Hypothesis: The unrolled version should perform better than the standard version

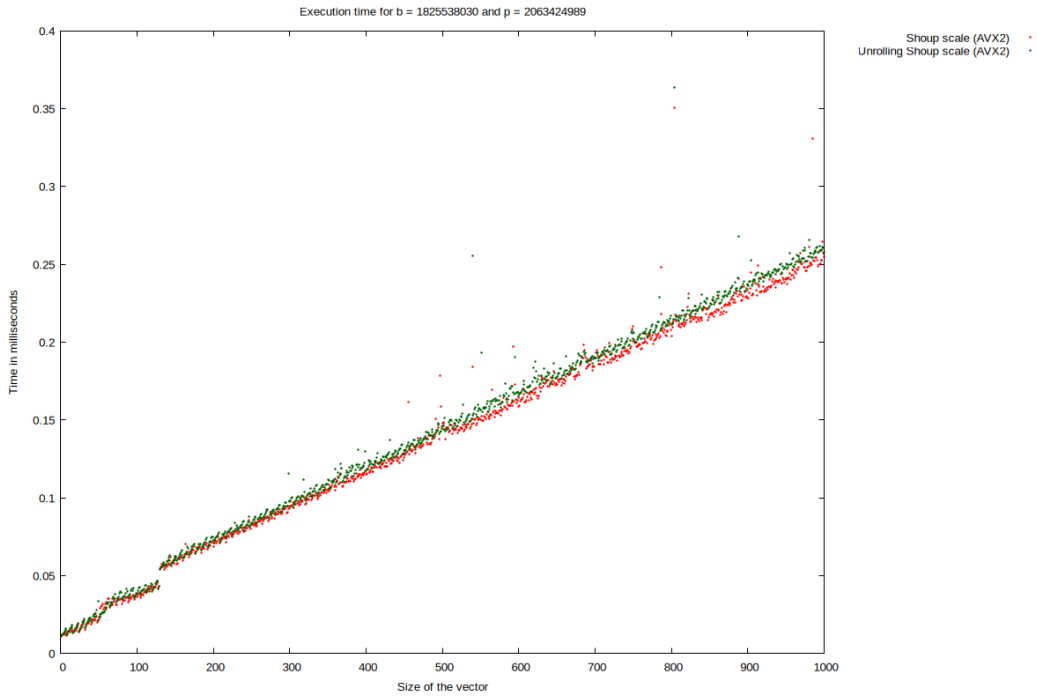

Figure 1: Execution Time Comparison: Standard vs. Unrolled

Size	Standard	Unrolled	Percentage difference
100	0.037	0.039	+5.41%
200	0.069	0.071	+2.90%
300	0.093	0.098	+5.38%
400	0.115	0.118	+2.61%
500	0.148	0.146	−1.35%
600	0.159	0.166	+4.40%
700	0.195	0.190	−2.56%
800	0.204	0.212	+3.92%
900	0.228	0.237	+3.95%
1000	0.255	0.258	+1.18%

Table 1: Execution time of implementations with and without unrolling in milliseconds

Observation: On average, the Unrolled implementation is 2.58% slower than the standard version.

Conclusion: We expected to see a bigger performance gap favoring the unrolled version but from the results obtained, this doesn't seem to be the case. We can conclude that an unrolled version for our implementation of Shoup's Modular Multiplication is not favorable.

3.9.5 Choice between mullo_epi32 and mul_epu32

mullo_epi32 and mul_epu32 both produce at max 64-bit results, the key difference is the former operation only truncates the results and only stores the 32 lower bits of the results whereas the latter stores the full 64-bit results.

The results below were obtained by measuring the execution time between the two variations described in Section 3.8.1

Hypothesis: The variant mullo_epi32 should perform better than the variant mul_epu32 as it uses fewer operations overall.

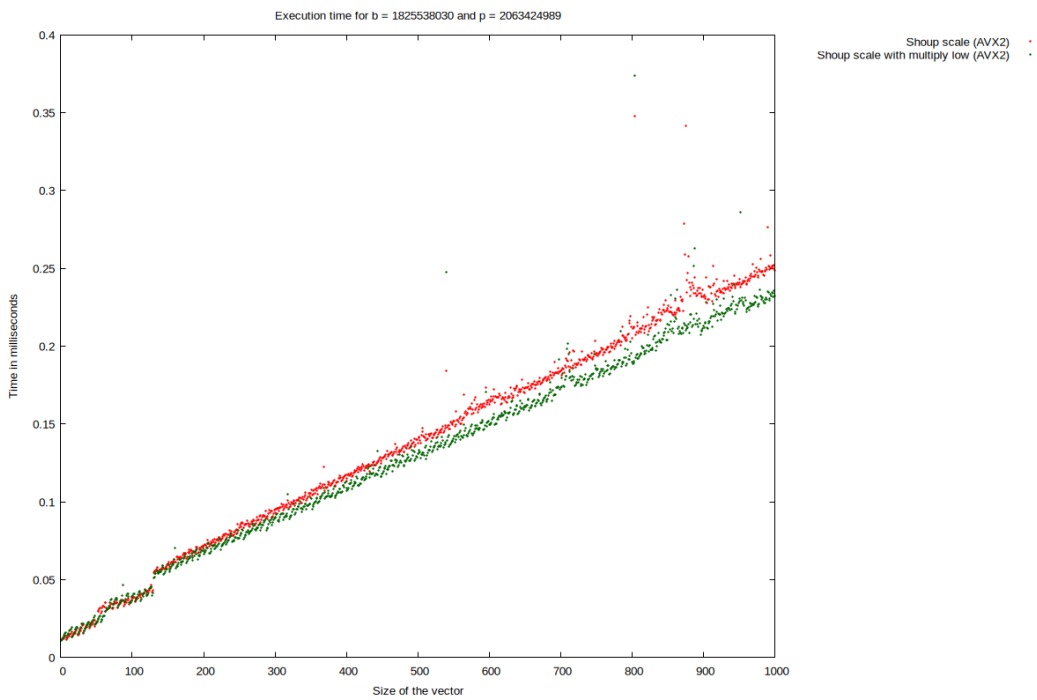

Figure 2: Execution Time Comparison: mullo_epi32 vs. mul_epu32

Size	mul_epu32	mullo_epi32	Percentage difference
100	0.037	0.038	+2.70%
200	0.069	0.066	−4.35%
300	0.093	0.090	−3.23%
400	0.113	0.107	−5.31%
500	0.138	0.131	−5.07%
600	0.163	0.150	−7.98%
700	0.184	0.173	−5.98%
800	0.205	0.190	−7.32%
900	0.232	0.213	−8.19%
1000	0.249	0.232	−6.83%

Table 2: Execution time of mullo_epi32 and mul_epu32 implementation in milliseconds

Observation: We can observe that at the smaller sizes (100), mullo_epi32 is slower than the standard mul_epu32 implementation by around 2%. But the efficiency gain starts to show once the vector size hits 200 and above. Across all the vector sizes, the average percentage gain is 5.15%

Conclusion: The mullo_epi32 version is effectively faster than the mul_epu32 version by about 5%, this speedup is especially seen for bigger vectors sizes. The mullo_epi32 version is the better implementation

3.9.6 Versions of mullo:

Both versions use mullo as the main multiplication operation, the key difference between these two versions is when the regrouping happens for the vector registers.

The results below are obtained by measuring the execution time between two implementations using `_mm256_mullo_epi32` described in Section 3.8.1

Hypothesis: The second version implementation should perform better, it carries out less redundant code by regrouping early the vector registers

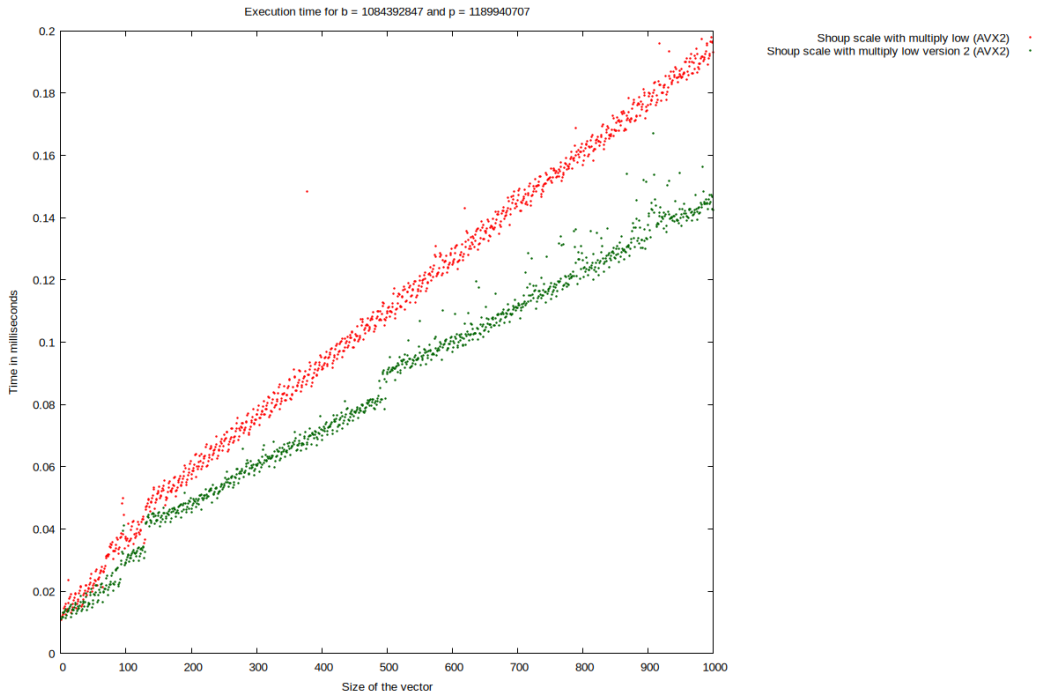

Figure 3: Execution Time Comparison between the two versions of multiply low

Size	Version 1	Version 2	Percentage difference
100	0.037	0.029	−21.62%
200	0.056	0.045	−19.64%
300	0.076	0.060	−21.05%
400	0.091	0.068	−25.27%
500	0.109	0.089	−18.34%
600	0.125	0.096	−23.2%
700	0.145	0.110	−24.13%
800	0.156	0.126	−19.23%
900	0.176	0.137	−22.15%
1000	0.193	0.142	−26.42%

Table 3: Execution time of the two versions of multiply low implementation in milliseconds

Observations:

Conclusion:

3.9.7 Overall Performance

It is worth noting that AVX2 uses 256-bit registers to process up to eight 32-bit integers, while AVX-512 uses 512-bit registers for up to sixteen. Furthermore, AVX-512 uses the same functions and operations as AVX2.

The results below were obtained by measuring the execution time of different implementations of modular reduction:

- **Naive scale:** The base operation (%)
- **Shoup scale (reference) :** Shoup's Modular Multiplication without optimization
- **Shoup scale (FLINT) :** the library FLINT's modular reduction implementation
- **Shoup scale (AVX2) :** Shoup's Modular Multiplication with AVX2 optimization
- **Shoup scale (AVX-512) :** Shoup's Modular Multiplication with AVX-512 optimization

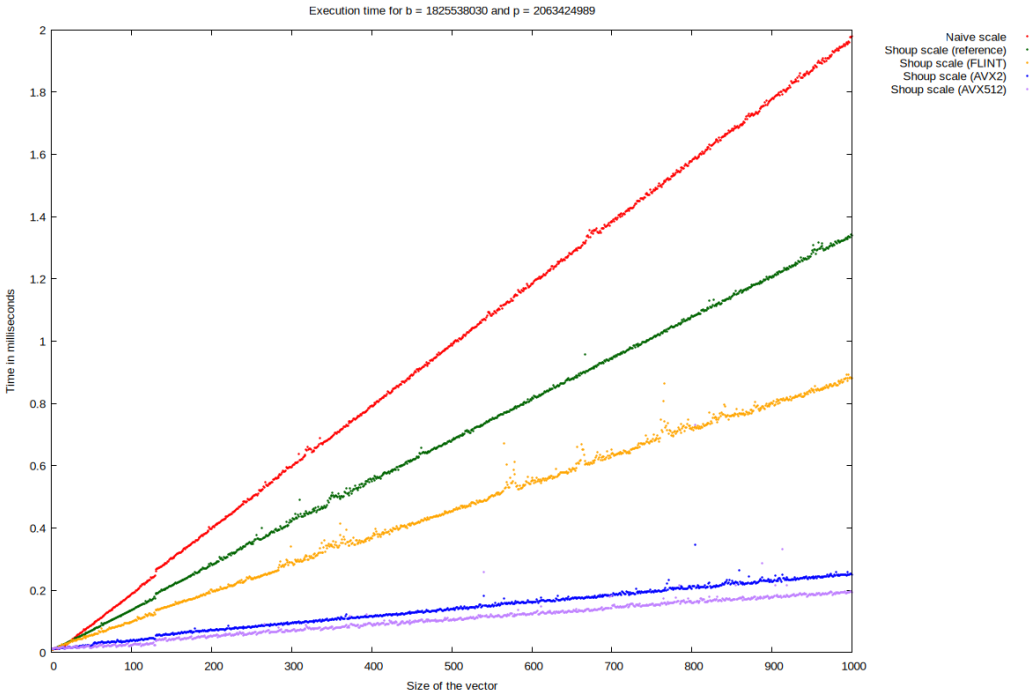

Figure 4: Execution Time Comparison Between Different Implementation For Modular Reduction

Size	Naive	Shoup ref	FLINT	Shoup AVX2	Shoup AVX-512
100	0.187	0.137	0.098	0.037	0.023
200	0.398	0.283	0.201	0.070	0.052
300	0.596	0.422	0.290	0.094	0.072
400	0.794	0.549	0.366	0.116	0.087
500	0.986	0.684	0.455	0.138	0.105
600	1.183	0.816	0.551	0.159	0.123
700	1.383	0.945	0.637	0.184	0.183
800	1.578	1.081	0.718	0.204	0.159
900	1.778	1.204	0.801	0.228	0.179
1000	1.979	1.339	0.880	0.249	0.193

Table 4: Execution time for different implementations in relation to the size of the vector in milliseconds

Table 5: Relative Performance Summary in Percentages

Algorithm	Average %	1x
Naive	100.0%	Baseline
Shoup ref	68.7%	1.45x
FLINT	45.6%	2.2x
Shoup AVX2	15.4%	6.5x
Shoup AVX-512	11.4%	8.7x

Observation: From the results retrieved, we can observe that FLINT's implementation over 64 bits provides roughly two times faster modular reduction on 32-bit operations. Furthermore, our Shoup AVX2 and Shoup AVX-512 implementations tailored to 32-bit integers provide significantly faster 6-8 times speedup.

Conclusion: Our Shoup's Modular Multiplication with AVX2 and AVX-512 optimization adapted to 32 bits is 2-3 times faster than FLINT.

3.9.8 NEON's Performance

TODO

4 Overall Conclusion:

For AVX architectures, we have researched and implemented many versions of Shoup's Modular Reduction. The best performance by far is the implementation *Intel* `_mm256_mullo_epi32` *version 2:* with acceleration of a factor of TODO for AVX2 and a factor of TODO for AVX512.

5 Other Improvements: